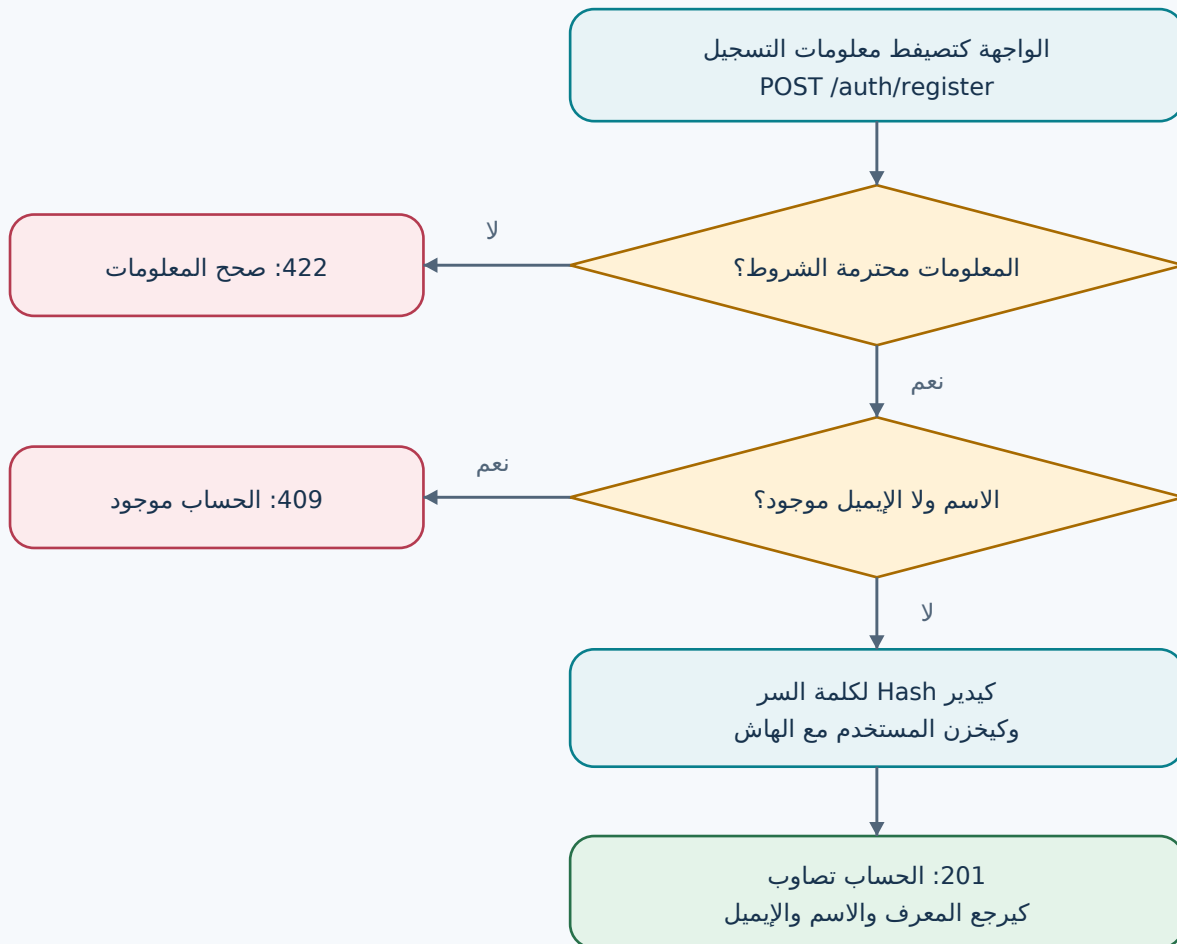


١. التسجيل العادي

إنشاء حساب جديد / POST /auth/register

الواجهة هي التي كيتعامل معاها المستخدم. السيرفر هو اللي كيتحقق من المعلومات. أول مرة، حمزة كيدخل الاسم والإيميل وكلمة السر.



مثال ديال المعلومات اللي كتصيفط الواجهة

```

POST /auth/register
{
  "username": "Hamza",
  "email": "hamza@example.com",
  "password": "HamzaLearning2026!"
}
  
```

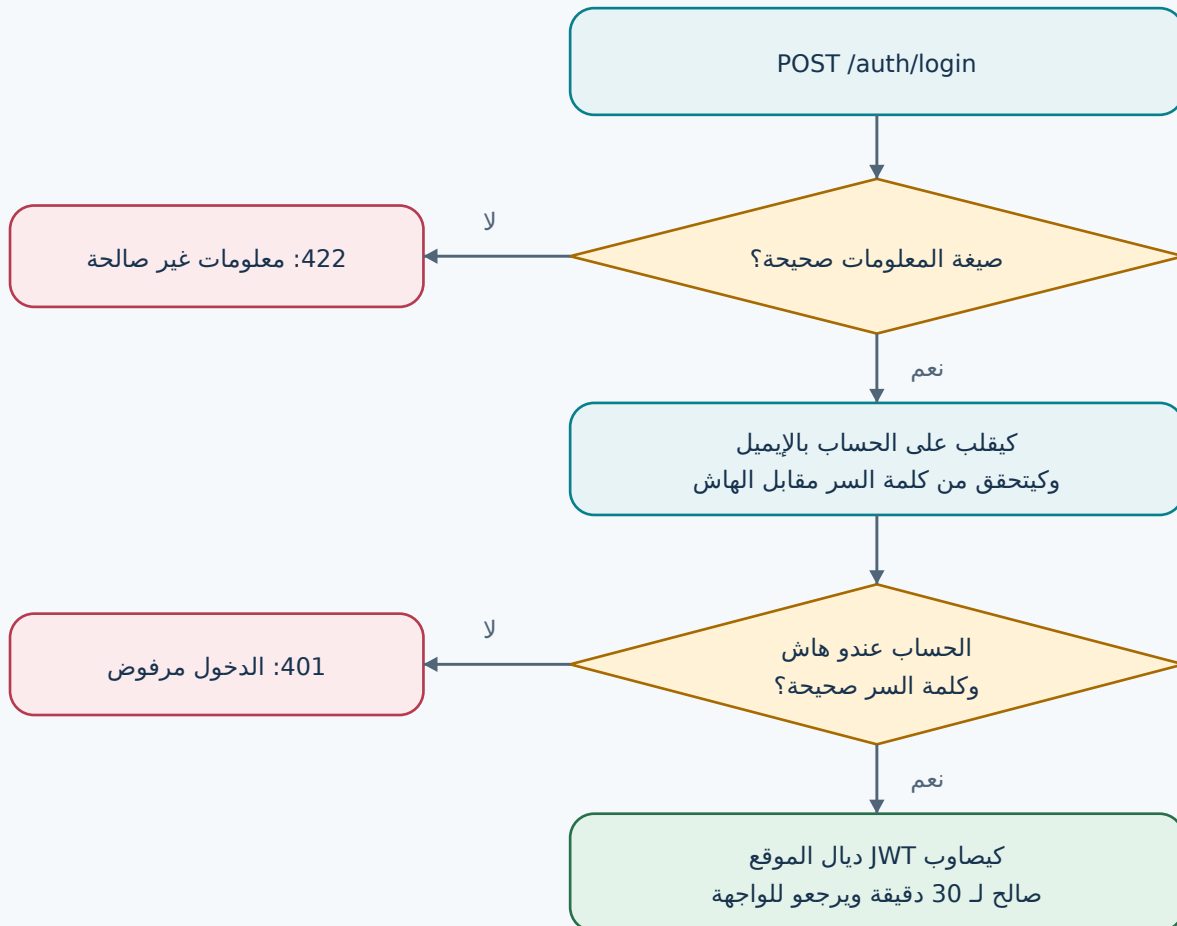
كلمة السر: من 15 حتى 128 حرف، فيها حروف كبار وصغار وأرقام ورموز، وبلا مسافات. الاسم: من 3 حتى 15 حرف، كيبدا بحرف وفيه غير حروف وأرقام.

كيخزن الهاش، ماشي كلمة السر الأصلية. الهاش ما كنرجعوش منو كلمة السر. التسجيل هنا ما كيرجعش JWT: خاص من بعد تدير Login.

٢. الدخول بالإيميل وكلمة السر

تأكيد الهوية / POST /auth/login

حمزة عندو حساب دابا. كيدخل الإيميل وكلمة السر، والسيرفر كيتأكد واش هاد المعلومات ديالو بصح.



POST /auth/login

```
{
  "email": "hamza@example.com",
  "password": "HamzaLearning2026!"
}
```

Response:

```
{
  "access_token": "SITE_JWT",
  "token_type": "bearer"
}
```

الدالة `verify_password` كتقارن كلمة السر اللي دخلتي مع الهاش المخزن باستعمال مكتبة الهاش. ما كتفكش الهاش باش تلقى كلمة السر.

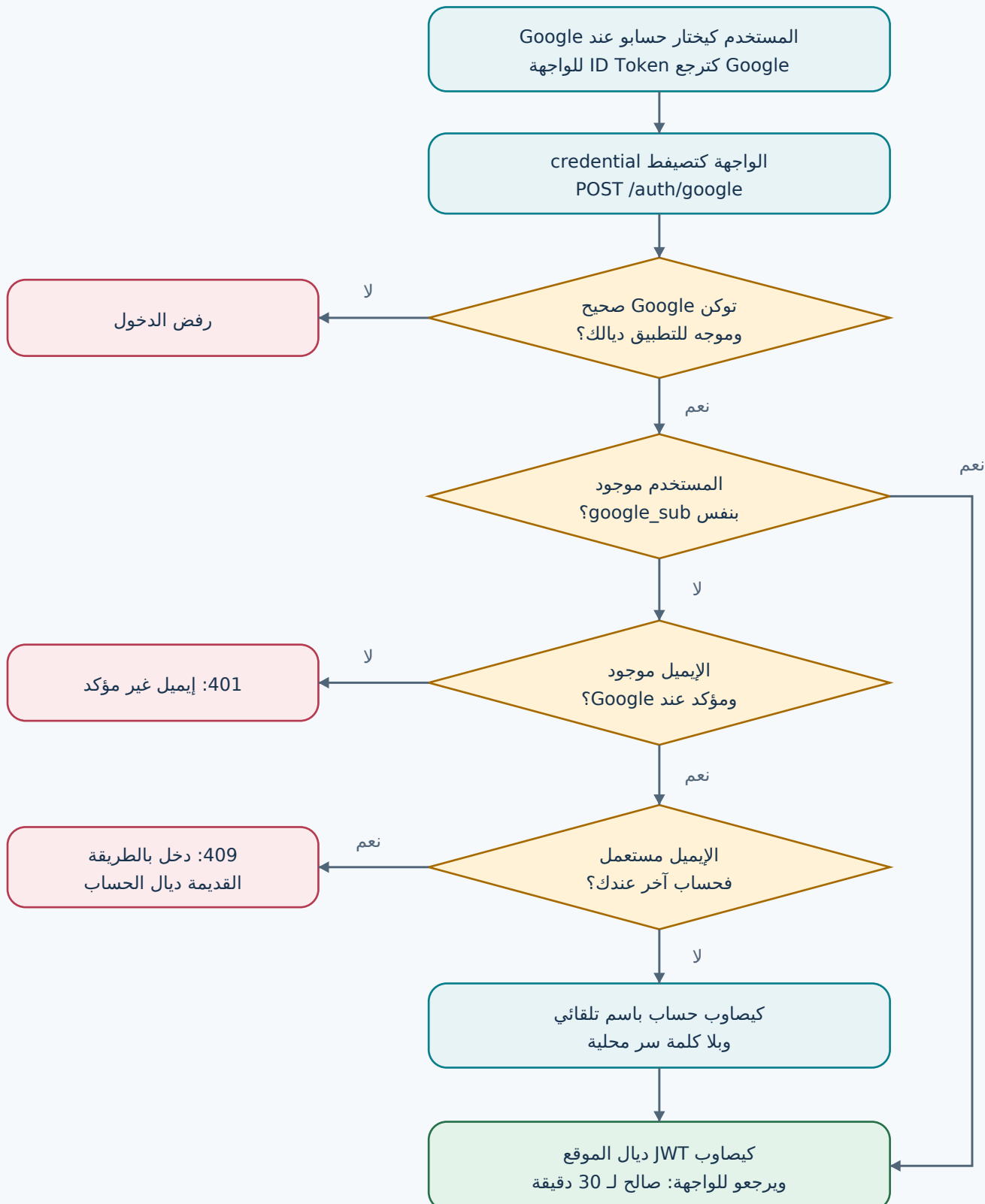
JWT بحال بطاقة دخول مؤقتة. فيها معرف المستخدم ف `sub`، وقت الإصدار ف `iat`، ووقت النهاية ف `exp`. السيرفر كيوقعها باستعمال السر ديالو؛ ما فيهاش كلمة السر.

عمليات الهاش والتحقق منها كتخدم ف `thread` بوحده، باش ما تحبسش الحلقة اللي كتسير الطلبات أثناء هاد الخدمة.

٣. الدخول بـ Google

من التحقق عند Google حتى JWT ديال الموقع

Google كتأكد الهوية ديال المستخدم. كلمة السر ديال Google ما كتوصلش للسيفر ديك.



مثال عملي: حمزة داخل بـ Google

جوج توكينات، وكل واحد عندو الدور ديالو

أول مرة حمزة كيضغط على زر Google، كيختار حسابو وكيكمل الدخول عند Google إلا كان خاص. الواجهة كتوصل بتوكن كيثبت الهوية ديالو.

```
POST /auth/google
Content-Type: application/json

{ "credential": "GOOGLE_ID_TOKEN" }
```

السيرفر كيتحقق من التوكن والتوقيع والصلاحيّة، وواش موجه لنفس GOOGLE_CLIENT_ID. من بعد كيشوف sub: هادا هو المعرف الثابت ديال المستخدم عند Google.

إلا ما لقا حتى حساب بهاد google_sub، كيتأكد من الإيميل وواش ما مستعملش عندو. كيصاوب حساب بحال هاد المثال؛ المعرف المحلي كيتولد بوحده:

```
username      = "Playera1b2c3d4e"
email         = "hamza@example.com"
google_sub    = "GOOGLE_USER_ID"
password_hash = None
```

من بعد كييرجع توكن الموقع. المرة الجاية، كيلقى حمزة باستعمال google_sub وكيعطيه توكن جديد بلا ما يعاود يصاوب الحساب.

```
Response:
{ "access_token": "SITE_JWT", "token_type": "bearer" }
```

الفرق بين جوج التوكينات

- Google ID Token: كتصاوبو Google، وكتصيفطو لـ /auth/google/ باش السيرفر يثبت الهوية ديالك.

- JWT ديال الموقع: كيصاوبو السيرفر ديالك، وكتستعملو فالطلبات المحمية بحال /auth/me/.

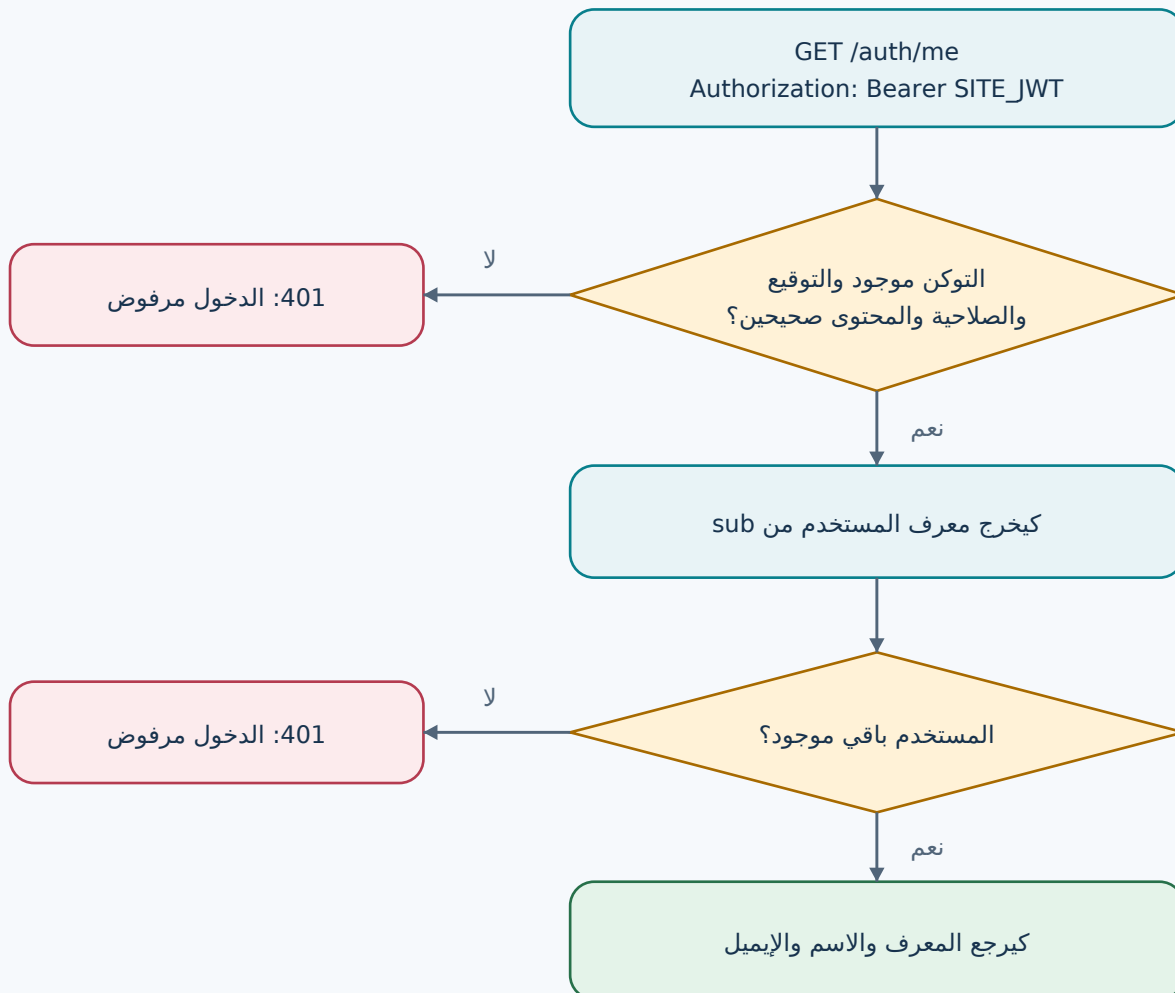
إلا كنتي تسجلتي من قبل بالإيميل وكلمة السر، وجربتي Google بنفس الإيميل، الكود كيرفض الربط التلقائي وكيطلب منك تدخل بالطريقة القديمة.

الملف index.html عندك غير للتجريب: كيعرض توكن Google باش تنسخو بيدك لـ docs/ وتحطو ف credential. ما كيرسلوش أوتوماتيكيا للسيرفر.

٤. من بعد الدخول: شكون نتا؟

نفس المسار سواء دخلتي بالطريقة العادية ولا بـ Google

باش يجيب حمزة معلومات الحساب ديالو، الواجهة كتصيفط JWT ديال الموقع فـ Authorization مع الطلب.



```
GET /auth/me
Authorization: Bearer SITE_JWT
```

Response:

```
{"id": "USER_UUID", "username": "Hamza",
  "email": "hamza@example.com"}
```

JWT كتصيفط مع كل طلب كيحتاج المصادقة، ماشي غير مرة وحدة. /auth/me كيخرج معلومات المستخدم، ما كيخرجش توكن جديد.

ملي كتسالي 30 دقيقة خاص تعاود تدخل: الكود المرفق ما فيهمش Refresh Token. وحتى التخزين ديال توكن الموقع فالواجهة ما محدش فهاد الملفات.

الحسابات دابا كتخزن فـ TEST_USERS فالذاكرة، ماشي فقاعدة بيانات دائمة. إلا تعاود شعلتي السيرفر، الحسابات كيتحيدو، والتوكن القديم ما يلقاش المستخدم.

مبني على الكود المرفق فـ backup.zip

كيفاش تقرا هاد الدليل؟

الصفحات 1 حتى 5: المخططات الأصلية | من هنا: الشرح بالتفصيل

الهدف هو تفهم الرحلة كاملة: شنو كيدخل المستخدم، شنو كيدوز فالواجهة، شنو كيتحقق منو السيرفر، وشنو كيتخزن وكيترجع. قرا المفاهيم، ومن بعد رجع للمخططات وتتبع الأسهم براسك.

خريطة القراءة

الصفحات	شنو غادي تلقى
1 - 5	التسجيل، الدخول العادي، Google، واستعمال التوكن: أربعة مخططات وأمثلة.
7 - 11	HTTP وFastAPI، الهاش والتوقيع، Cookie، JWT وSession، والصلاحيية والخروج.
12 - 14	OAuth 2.0 وOIDC، مسار Authorization Code، وasync وthreads.
15 - 22	شرح الملفات والدوال والكلاسات، مع المدخلات والنتائج والأخطاء والأمثلة.
23 - 25	تتبع عملي، حدود النسخة الحالية، أسئلة للفهم وأجوبتها.
26 - 27	المراجع الرسمية وروابطها القابلة للضغط.

ثلاثة أنواع ديال الشرح

«الكود ديالك»: سلوك موجود فعلا ف backup.zip. «مثال تعليمي»: معلومات وهمية باش بيان المعنى. «تطوير من بعد»: فكرة مازال ما مطبقةش فهاد الملفات.

هاد الدليل كيشرح النسخة المرفقة، ما كيبدلش كود المشروع. الأمثلة بحال SITE_JWT وUSER_UUID غير أسماء توضيحية؛ ماشي توكنات ولا معرفات صالحة للتجريب كما هي.

باش تتأكد من الفهم، حاول تشرح كل سهم فالمخططات بكلامك، وتوقع النتيجة ديال الأسئلة فالصفحة 24 قبل ما تشوف التصحيح.

الأساس: الطلب والجواب

FastAPI, Authentication, Authorization, HTTP, JSON

واش عرفتك؟ وواش مسموح ليك؟

Authentication هي إثبات الهوية: واش نتا فعلا حمزة؟ Authorization هي الصلاحيات: من بعد ما عرفناك، واش عندك الحق تمشح غرفة ديال شي واحد آخر؟ الدخول بوجدو ما كيغطيكنش الحق تدير كلشي.

شئو كيتصيفط فالطلب؟

```
POST /auth/login
Content-Type: application/json

{"email": "hamza@example.com",
 "password": "HamzaLearning2026!"}
```

POST هو نوع العملية. /auth/login/ هو المسار. Headers فيها معلومات على الطلب؛ Content-Type كيبين أن Body مكتوب بصيغة JSON. JSON نص منظم، كيتحول هنا لكائن Python من نوع LoginData؛ ماشي ضروري يكون ملف json فالديسك.

GET /auth/me كيطلب معلومات. الجواب فيه Status Code وBody. ف HTTPS، الاتصال كيتشفّر أثناء النقل؛ الهاش ديال التخزين ما كيغوضش HTTPS.

شكون كيغيط للدالة اللي تحت @ ؟

ال decorator بحال @login("auth_router.post") كيسجل العلاقة بين المسار والدالة. ملي كيوصل الطلب المناسب، FastAPI كيقرأ Body، كيتحقق من LoginData، ومن بعد كيغيط لـ login(data). نتا ما محتاجش تعيط ليها بيدك.

APIRouter كيجمع المسارات؛ prefix="/auth" كيزيد البداية المشتركة. خاص التطبيق الرئيسي يضم auth_router باستعمال include_router. الملف الرئيسي ما موجودش فالأرشيف، لذلك الربط مع التطبيق ما نقدرش نثبتو من هاد الملفات بوجدها.

Depends(get_current_user) كتعني: نفذ التحقق أولا، وخذ النتيجة وحطها ف user. إلا التحقق رفع خطأ، me ما كتوصلش ترجع المعلومات.

الهاش، التشفير والتوقيع

ثلاثة مفاهيم، وكل واحد كيجل بشكل مختلف

المفهوم	شنو كيدير؟	مثال فالمشروع
Hash	بصمة فجهة وحدة؛ ما كنفكوهاش باش نرجعو الأصل.	تخزين كلمة السر
Encryption	كيخبي المحتوى؛ اللي عندو المفاتيح المناسبة كيقدر يفكو.	HTTPS أثناء النقل
Signature / MAC	كيكشف واش المحتوى تبدل وواش جا من جهة عندها المفتاح.	JWT دبال الموقع

علاش ما نخزنوش كلمة السر مباشرة؟

إلا تسرب التخزين، كلمة السر الأصلية كتكون مكشوفة فوراً. هاش كلمات السر كيزيد كلفة التخمين. خاص خوارزمية مخصصة لكلمات السر؛ SHA-256 بوجدو سريع بزاف لهاد الغرض. المكتبة كتسير Salt عشوائي وإعدادات الكلفة. [S8]

Salt ماشي كلمة سر ثانية ولا سر خاصك تخبيه: قيمة عشوائية كتخلي حتى نفس كلمة السر تعطي هاش مختلف. كيتخزن غالباً داخل صيغة الهاش مع الإعدادات. verify كنقرا هاد المعلومات وتتحقق من كلمة السر المرشحة. [S8]

```
h1 = hash_password("HamzaLearning2026!")
h2 = hash_password("HamzaLearning2026!")
# h1 and h2 can differ because of random salts.
verify_password("HamzaLearning2026!", h1) # True
verify_password("WrongPassword!", h1)      # False
```

السر دبال JWT ماشي سر الهاش

فالكود دبالك، JWT_SECRET_KEY كيتستعمل لتوقيع JWT والتحقق منو بـ hash_password. HS256 ما كتستعملش هاد المتغير: كتعيط لـ password_hasher.hash. تغيير JWT_SECRET_KEY كخلي التوكنات القديمة تفشل، ولكن ما كيدلش هاش كلمات السر.

HS256 كيتستعمل نفس السر للتوقيع والتحقق: هادا HMAC متماثل. فالتوقيع غير المتماثل، جهة كتوقع بالمفتاح الخاص والآخرين كيتحققو بالمفتاح العام. التحقق من Google كيتكلف بيه Google library ومفاتيح Google العامة. [S1]

JWT: شنو كاين داخل التوكن؟

JSON Web Token | المثال هنا JWT موقع، ماشي مشفر

JWT صيغة كتجمع معلومات كتسمى Claims. الشكل الموقع الشائع فيه ثلاثة أجزاء مفروقين بنقطة: Header و Payload و Signature. كصيغة يقدر يكون مشفر كذلك، ولكن jwt.encode بـ HS256 فالكود دياك كيقوع فقط. [S6]

HEADER.PAYLOAD.SIGNATURE

Illustrative decoded payload:

```
{
  "sub": "550e8400-e29b-41d4-a716-446655440000",
  "iat": 1800000000,
  "exp": 1800001800
}
```

Header فيه نوع وخوارزمية التوكن. Payload فيه المعلومات. التوقيع مرتبط بالأجزاء السابقة: إلا بدل شي واحد sub باش ينتحل مستخدم آخر، التوقيع ما غاديش يوافق. Base64url ترميز يمكن تقراه؛ ماشي وسيلة لإخفاء المحتوى. [S6]

الحقل	الدور فالكود دياك
sub	معرف المستخدم المحلي من نوع UUID، ماشي الإيميل.
iat	وقت الإصدار. الكود كيستعمل الوقت الحالي بـ UTC.
exp	وقت انتهاء الصلاحية: بعد 30 دقيقة من الإصدار.
nbf	وقت بداية القبول إلا كان موجود؛ الكود ما كيضيفوش عند الإنشاء.

مثال: توكن تصاوب مع 10:00، كيسالي مع 10:30. طلب صحيح مع 10:10 يدوز؛ مع 10:31 يترفض. إرسال التوكن لـ me/ ما كيزيدش الوقت دياك وما كيجددوش.

Bearer كتعني أن حيازة التوكن كتسمح باستعمالو إذا كان صالح وباقي الشروط متوفرة. اللي سرقو يقدر يستعملو حتى يسالي ولا يتلغى بمنطق إضافي. ما تحطش فيه كلمة السر، وما تسجلش التوكنات ف logs.

Session و Cookie و JWT

ماشي ثلاثة بدائل من نفس النوع

JWT هو صيغة ديال التوكن. Cookie وسيلة باش المتصفح يخزن قيمة ويصيفطها تلقائيا فطلبات كتوافق قواعدها. Session هي الطريقة اللي كنهتافطو بها بحالة الدخول بين الطلبات. تقدر Cookie تحمل JWT، وتقدر تحمل غير [Session ID. [S4, S10

المقارنة	JWT	Session ID
المحتوى	Claims وتوقيع داخل التوكن.	معرف عشوائي، المعنى ديالو فالتخزين ديال السيرفر.
التحقق	السيرفر كيتحقق من التوقيع والشروط.	السيرفر كيقلب على السيشن المسجلة.
الإلغاء	خاص منطق إضافي إلا بغينا نلغيوه قبل exp.	حذف السيشن من التخزين كيوقف استعمالها.
النقل	يقدر يدوز ف Header ولا Cookie.	يقدر يدوز ف Cookie.

مثال السيشن: السيرفر عندو الدفتر

```
# Server-side session store (illustration)
sessions["random_session_id"] = {"user_id": "USER_UUID"}

# Response header
Set-Cookie: sid=random_session_id; HttpOnly; Secure; SameSite=Lax

# Later request header
Cookie: sid=random_session_id
```

المتصفح كيصيفط sid، والسيرفر كيقلب عليه باش يعرف المستخدم. القيمة فالمتصفح ما محتاجاش تحتوي الاسم ولا معلومات الحساب. [S10]

والمشروع ديك؟

السيرفر كيخرج JWT داخل get_current_user. JSON كتنقراه من Authorization: Bearer. ما كين لا Set-Cookie، لا قراءة Cookie، لا جدول ديال Sessions فهاد الملفات. إلا حطيتي JWT غير ف Cookie، هاد النسخة ما غاديش تنقراه بلا تعديل.

التوقيع بوحده ما محتاجش تخزين التوكن، ولكن كودك كيقلب على المستخدم مع كل طلب. لذلك ماشي دقيق نقولو النظام كامل ما عندوش حالة مخزنة.

فين كيبقى التوكن؟ وشنو هو Logout؟

التخزين، XSS، CSRF، والصلاحيه

الاختيار فالواجهة	شنو خاص تفهم
Memory	متغير ف JavaScript؛ كيتحدد ملي كتعاود تحمل الصفحة.
localStorage	كيبقى بين الزيارات، و JavaScript ديال نفس origin يقدر يقرأه.
HttpOnly Cookie	JavaScript ما يقدرش يقرأ القيمة؛ المتصفح يقدر بصيفطها تلقائيا.

هاد الخيارات ماشي مطبقة فواجهة الموقع الكاملة داخل الأرشيف. اختيار التخزين كيتحتاج يتوافق مع تصميم الواجهة وال localStorage، API، ما كيبغش Authorization بوحده؛ كود الواجهة هو اللي كيصيفو. [S14]

HttpOnly كتمنع قراءة الكوكي من JavaScript؛ Secure كتنقيد إرسالها ل HTTPS فالاستخدام العادي؛ SameSite كتنقيد بعض الطلبات اللي جاية من سياق موقع آخر. هاد الإعدادات ما كتجعلش التطبيق محصن بوحدها. [S9]

XSS و CSRF بأمثلة بسيطة

XSS: كود JavaScript خبيث قدر يخدم داخل موقعك، مثلا بسبب عرض محتوى غير موثوق بلا حماية. يقدر يقرأ توكن مكشوف ل JavaScript، أو يدير طلبات باسم المستخدم حتى مع HttpOnly. خاص حماية عرض المحتوى.

CSRF: موقع خبيث كيحاول يخلي المتصفح يدير عملية فموقعك باستعمال كوكي الدخول اللي كنتصيفط تلقائيا. الحماية كتحتاج تصميم مناسب بحال CSRF token والتحقق من Origin و CORS و SameSite. بوحدها ماشي نظام مصادقة ولا حماية شاملة من [S11]. [CSRF]

الخروج والتجديد فهاد النسخة

حذف JWT من الواجهة كيخرج المستخدم من هديك الواجهة. ولكن نسخة مسروقة منو تقدر تبقى مقبولة حتى exp: ما كاينش Logout endpoint ولا blacklist ولا session revocation فالكود المرفق.

Refresh Token توكن منفصل كيتمكن يعطي Access Token جديد؛ كيتحتاج تخزين وحماية وإلغاء مناسبين. ما كاينش هنا، لذلك من بعد 30 دقيقة خاص إعادة الدخول. تغيير كلمة السر بوحده حتى هو ما كيلغيش JWT قديم فالتصميم الحالي.

OAuth و OAuth 2.0 و OIDC

الصلاحيات والهوية: فين كاين الفرق؟

OAuth اسم عائلة ديال بروتوكولات تفويض الصلاحيات. ملي شي واحد كيقول OAuth، يقدر يقصد العائلة عامة، وماشي بالضرورة النسخة 1.0. OAuth 2.0 هي نسخة مختلفة ومش متوافقة مباشرة مع 1.0. [S2، S3]

مثال: تطبيق بغا يقرأ صورك من خدمة أخرى. كتوافق عند هديك الخدمة على صلاحية محددة؛ التطبيق كياخذ Access Token، وما كتعطيهش كلمة السر دبالك. Scope كيبيّن حدود الإذن، بحال القراءة فقط. [S3]

المقارنة	OAuth 1.0a	OAuth 2.0
الفكرة	بروتوكول أقدم لتفويض الوصول.	إطار تفويض فيه مسارات مختلفة.
الطلبات	الطرق الشائعة كتوقع الطلبات بمعطيات ومفاتيح العميل والتوكن.	استعمال Bearer شائع مع HTTPS؛ كابين امتدادات أخرى.
التوافق	ما تقدرش تبدلو لـ 2.0 غير بتغيير رقم.	ماشي مجرد تغيير الاسم؛ التنفيذ مختلف.

فين كيدخل OpenID Connect؟

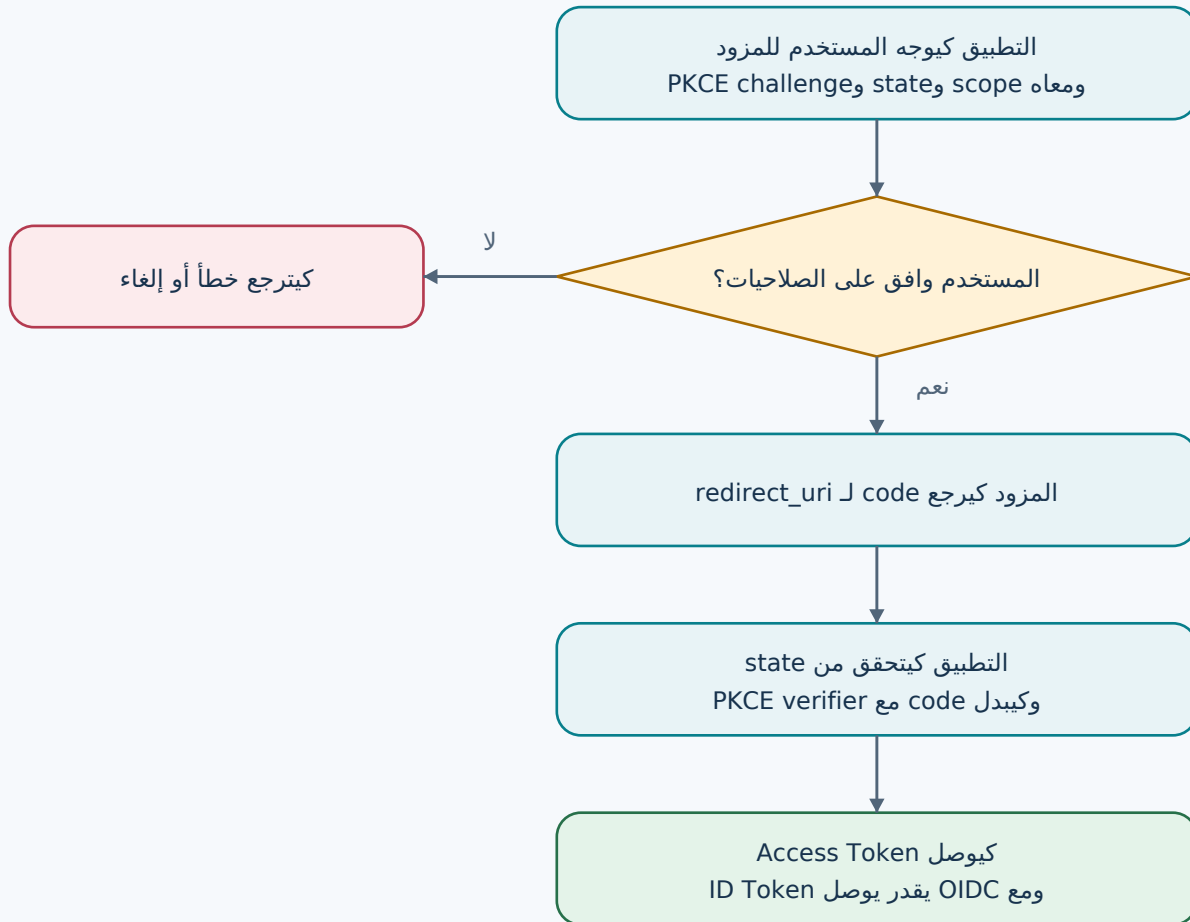
OIDC طبقة ديال إثبات الهوية مبنية فوق OAuth 2.0. كتحدد ID Token باش التطبيق يعرف شكون دخل. OAuth 2.0 بوجدو ما كيعطيش عقد موحد لإثبات هوية المستخدم؛ Access Token ديال API ما خاصوش يتخلط مع ID Token. [S5]

النوع	شنو كيبرهن أو يسمح؟
ID Token	هوية المستخدم للتطبيق اللي التوكن موجه ليه.
Access Token	الوصول لموارد API فحدود الصلاحيات المعطاة.
Refresh Token	طلب Access Token جديد حسب النظام.

فالكو دبالك، Google Identity Services كترجع ID Token، والسيرفر كيتحقق منو وكيصدر Access Token محلي. هادشي ما كيعطيش موقعك تلقائيا الحق يقرأ Gmail ولا Google Drive.

Authorization Code: مسار آخر

مثال للتعليم فقط؛ ماشي المسار المطبق فالأرشيف



Authorization Code رمز قصير العمر كيتبدل بتوكنات؛ ماشي JWT ديال موقعك. `redirect_uri` هو عنوان الرجوع المسجل. `state` كييربط الجواب بالطلب اللي بدا من عندك، وكيعاون ضد خلط الطلبات. [S3]

PKCE كيولد `verifier` عشوائي لكل محاولة، وكيصيفط فالبداية `challenge` مشتق منو. فالتبديل كيثبت التطبيق أنه عندو نفس `verifier`، باش سرقة `code` بوحدها ما تكفيش. [S7]

OIDC، `nonce` يقدر يربط ID Token بالمحاولة وبتحقق منو التطبيق. Client ID هو معرف التطبيق وماشى سر. Client Secret، إلا استعملو تصميم فيه Backend سري، ما يتحطش فالواجهة. [S5]

ف `index.html` ديالك، `callback` كتستقبل `response.credential` مباشرة من Google library. ما كاينش `code` exchange ولا `redirect endpoint` ولا Client Secret فالكود المرفق؛ ما تزيدش هاد المراحل فشرح التدفق الفعلي.

Thread, await, و async

علاش كنستناو الهاش بلا ما نحيسو الحلقة ديال الطلبات؟

def كتعرف دالة عادية. async def كتعرف coroutine: ملي كتعيط ليها كتحصل على coroutine خاصها تنفذ بـ await أو async scheduling. بوحدها ما كتحولش الخدمة الثقيلة لشي حاجة خفيفة.

await كتوقف تقدم هاد الطلب حتى توصل النتيجة، ولكن فعملية انتظار مناسبة كتخلي Event Loop تخدم طلبات أخرى. ما كتعنيش أن المستخدم توصل بالجواب قبل ما يسالي الهاش. [S13]

```
password_hash = await run_in_threadpool(
    hash_password,
    data.password.get_secret_value(),
)
# Only this request waits for its hashing result.
```

run_in_threadpool كتخط الدالة العادية ف worker thread. الـ await كيتسنى النتيجة بطريقة ما كتحسش Event Loop. الدالة والباراميترات كيتعطاو منفصلين؛ ما كنكتبوش hash_password (...) داخل الوسيط الأول ونحسبوها قبل الإرسال للـ thread.

مثال بجوج طلبات

الوقت التقريبي	شئو واقع؟
المرحلة 1	طلب A بدا التسجيل وصيفط الهاش لـ thread.
المرحلة 2	A كيتسنى؛ Event Loop تقدر تخدم طلب B ديال /me.
المرحلة 3	الهاش ديال A سالا؛ A كيكمل إنشاء الحساب ويرجع الجواب.

فالكود دبالك كاين هاد النمط فالـ register، فالـ authenticate_user للتحقق من الهاش، وفـ google_login للتحقق من توكن Google اللي يقدر يحتاج اتصال خارجي.

Thread pool عندو حدود وكيتسهلك CPU و RAM؛ ماشي قدرة لا نهائية وماشي ضمان السرعة مع آلاف الطلبات. الهدف هنا ما نخليوش العملية العادية تحبس الحلقة الرئيسية.

FastAPI يقدر يشغل route مكتوبة بـ def ف thread pool. ولكن دالة def كتعيط ليها مباشرة من داخل async def ما كتخرجش تلقائيا لشي thread: خاص تكون عارف فين كتخدم. [S13]

schemas.py: شكون كيتقبل؟

كلاسات الطلب والجواب، وكل دالة ديال التحقق

SecretStr, Field و BaseModel

BaseModel كيغطي نموذج كيتحقق من البيانات. Field كيزيد قيود بحال الطول. EmailStr كيتحقق من صيغة الإيميل؛ ما كيثبتش أن المستخدم مالك هاد الإيميل. SecretStr كيخبي القيمة فالعرض العادي، ما كيدير لا Hash لا Encryption. get_secret_value () كترجع النص الأصلي. [S15]

LoginData و RegisterData

RegisterData: email حتى 256 حرف، username من 3 حتى 15، وكلمة السر من 15 حتى 128 مع شروط التعقيد. LoginData: email حتى 256، وكلمة السر من 1 حتى 128. فالدخول ما كنعيدوش فرض سياسة إنشاء كلمة سر جديدة؛ كنقارن اللي عطانا مع المخزن.

validate_password_encoding(value)

كتستقبل SecretStr، كنجرّب تحويل النص لـ UTF-8، وكترجع نفس القيمة إذا نجح. إلا فشل كتطلع ValueError. ما كتبدلش كلمة السر وما كتخزنهناش. Password = Annotated [...] كيجمع النوع وAfterValidator اللي كيشغل هاد التحقق.

validate_username(cls, username)

كتشوف أول حرف بـ isalpha، والاسم كامل بـ isalnum. كتقبل اسم بحال Hamza42، وترفض 42Hamza ولا Hamza_42. هاد دوال Python كتقبل حروف Unicode حتى هي، ماشي غير A-Z. إلا صحيح كترجع الاسم كما هو.

validate_password_complexity(cls, value)

كتخرج النص من SecretStr وكتطبق password_policy: حرف كبير وصغير ورقم ورمز وبلا مسافات. إلا فشل كترفع ValueError، وإلا نجح كترجع SecretStr نفسها. field_validator كييربط التحقق بالحقل، وclassmethod كتمرر cls بلا ما تحتاج جاهزة.

GoogleLoginData و TokenResponse و UserResponse

UserResponse كيخرج غير id وusername وemail. TokenResponse كيخرج access_token وtoken_type اللي افتراضيا bearer. GoogleLoginData ف google_auth.py كتطلب credential من نوع SecretStr وبطول واحد على الأقل؛ هادشي ما كيثبتش صحة Google token حتى تدوز التحقق الحقيقي.

router.py: الأبواب ديال التطبيق

login, register(), me()

الإعداد ديال Router

APIRouter(prefix="/auth", tags=["AUTH"], route_class=AuthRoute) كيجمع المسارات تحت /auth، وكنظمهم فالتوثيق تحت AuthRoute. AUTH. كيخصص كيفاش ترجع أخطاء التحقق. include_router(google_router) كيخلي /google تولي /auth/google.

register(data: RegisterData) → UserResponse

المدخلات: بيانات تسجيل سبق دازت التحقق. أول حاجة كيقلب على الاسم أو الإيميل. إلا موجود كيرفع 409. إلا ما موجودش، كيحسب الهاش ف thread، ومن بعد create_user كتعاود تفحص التكرار وكتخزن المستخدم.

علاش التحقق جوج مرات؟ أثناء انتظار الهاش، طلب آخر يقدر يسجل نفس الاسم. الفحص الثاني كيلتقط هاد الحالة فهاد التصميم. فقاعدة بيانات حقيقية خاص Unique Constraints وعمليات مناسبة؛ فحص Python بوحده ماشي ضمان ضد جميع حالات التزامن.

النجاح: 201 ومعه id و username و email فقط. ما كيرجع لا password_hash ولا create_user. JWT. تقدر ترفع DuplicateUserError، و register كتحولها لخطأ HTTP 409.

login(data: LoginData) → TokenResponse

كتنخرج كلمة السر الأصلية باش تدوز ل authenticate_user. إلا رجعات None، كترد 401 برسالة عامة Invalid email or password ومع WWW-Authenticate: Bearer. إلا رج user، كتنخلق JWT من user["id"] و كتردو للواجهة.

```
user = await authenticate_user(email, password)
if user is None:
    raise HTTPException(status_code=401)
token = create_access_token(user["id"])
# Return TokenResponse(access_token=token)
```

me(user = Depends(get_current_user))

ما كتستقبلش user_id من Body و تصدقو. FastAPI كينفذ get_current_user ويحقن المستخدم اللي ثبتت هويتو. من بعد me كترجع نفس حقول UserResponse. هاد التصميم كيمنع المستخدم من اختيار هوية أخرى غير بتغيير Body ديال الطلب.

security.py: الهاش والدخول

`authenticate_user()` و `hash_password()`، `verify_password()`

الإعدادات التي كيتقراو ملي كيتحمل الملف

JWT_SECRET_KEY كيتقرا من متغيرات البيئة. إلا ما موجودش، RuntimeError كتوقف تحميل هاد الجزء بدل استعمال سر افتراضي. JWT_ALGORITHM هو HS256، و ACCESS_TOKEN_EXPIRE_MINUTES هي 30. password_hasher كيتصاوب ب PasswordHash.recommended().

hash_password(password: str) → str

كتاخذ كلمة سر نصية، كتعيط ل password_hasher.hash، وكترجع هاش نصي. ما كتسجل حتى user بوحدها. register هي التي كتستعمل النتيجة ف create_user. نوع وإعدادات الهاش تابعين للمكتبة المثبتة، والكود ما كيكتبهمش يدويا.

verify_password(password, password_hash) → bool

كتاخذ كلمة السر المرشحة والهاش المخزن، وكتطلب من المكتبة التحقق. النتيجة True أو False. إلا صيغة الهاش غير معروفة ووقع UnknownHashError، كترجع False. ماشي catch لجميع الأخطاء الممكنة ديال البرنامج.

DUMMY_PASSWORD_HASH: علاش كاين؟

هاش ثابت محسوب عند تحميل الملف لكلمة تجريبية. إلا الإيميل ما موجودش أو الحساب ديال Google بلا هاش، الكود ما كيقفزش مباشرة للرفض؛ كيدير verify بهاد الهاش الوهمي باش ينقص الفرق الواضح فالوقت. هادشي كيقلل تسريب وجود الحساب عبر التوقيت، ما كيضمنش توقيت متطابق تماما.

authenticate_user(email, password) → user أو None

(1) كتقلب بالإيميل. (2) كتاخذ password_hash إلا كان 3. user) كتدير verify ف thread بالهاش الحقيقي أو (4. DUMMY) إلا ما كاينش user، أو هاش، أو المطابقة فاشلة، كترجع None. إلا كلشي صحيح كترجع قاموس المستخدم.

```
# A Google-only account has password_hash = None.
# Even if dummy verification matched, this check rejects it:
if user is None or not password_hash or not password_matches:
    return None
```

هاد الشرط الأخير مهم: الهاش الوهمي ما كيخلقش طريق للدخول. حتى إلا طابقت كلمة التجريب، الحساب بلا هاش حقيقي كييقى مرفوض فالدخول العادي.

current_user.py و security.py

إنشاء التوكن، التحقق منو، واستخراج المستخدم

create_access_token(user_id: str) → str

كتحسب now بـ UTC، كتصاوب payload فيه sub وiat وexp، وكتعيط لـ jwt.encode مع JWT_SECRET_KEY وHS256. كترجع التوكن كنص. ما كتزنوش وما كتبعثوش للمستخدم بوحدها؛ login أو google_login هما اللي كيرجوه ف Response.

decode_access_token(token: str) → user_id

الاسم decode ما كيغيش هنا غير قراءة النص: jwt.decode كتحقق من التوقيع بالخوارزمية المسموحة، والصلاحيه، ووجود sub وiat وexp. أخطاء المكتبة الخاصة بالتوكن كتوصل للمنادي. بعض أخطاء الأنواع والقيم كتتحول صراحة لـ InvalidTokenError.

من بعد الكود كيفرض أن iat وexp وnbف إلا موجودة تكون integers، وsub تكون str بصيغة UUID صالحة. كيرجع غير user_id. هاد الدالة ما كتقلبش واش الحساب موجود فالتخزين، وما كتبتش صلاحيات المستخدم.

HTTPBearer(auto_error=False)

كيقرا Authorization ويستخرج بيانات Bearer. auto_error=False كيخلي غياب credentials يرجع None باش الكود يعطي نفس رسالة الرفض المخصصة. credentials.credentials هي النص ديال JWT بلا كلمة Bearer.

unauthorized() → HTTPException

كتصاوب exception جاهزة: 401 ومع WWW-Authenticate: Bearer. هي كترجع كائن الخطأ فقط؛ raise unauthorized() هي اللي كترفعو وتوقف المسار.

get_current_user(credentials) → StoredUser

إلا credentials غائبة: 401. إلا decode_access_token رفعات 401. InvalidTokenError: إلا التوكن صحيح، كتقلب بـ find_user_by_id. إلا المستخدم ما موجودش: 401. غير فحالة النجاح كترجع user لـ me عبر Depends.

```
Authorization: Bearer SITE_JWT
```

```
HTTPBearer -> decode_access_token -> find_user_by_id
          -> StoredUser -> me -> UserResponse
```

from None فرقع بعض الاستثناءات كيخبي ربط الاستثناء السابق فالتتبع المعروض ديال Python؛ ما كيغيش أن القيمة الراجعة هي None.

memory_store.py: التخزين

كل دالة كتقلب على شنو؟ وشنو كترجع؟

DuplicateUserError, StoredUser, TEST_USERS

TEST_USERS لائحة ف RAM. TypedDict كيوصف مفاتيح القاموس للمساعدة ف typing؛ ماشي Pydantic validator ولا قاعدة بيانات. NotRequired كتعني المفتاح يقدر يغيب، وNone كتعني يقدر يكون حاضر ولكن بلا قيمة. DuplicateUserError exception مخصصة للتكرار.

الدالة	شنو كتقلب عليه؟	شنو كترجع؟
find_existing_user_id	الاسم أو الإيميل بلا فرق فحالة الحروف.	id أو None
find_user_by_email	الإيميل باستعمال casefold.	user أو None
find_user_by_id	المعرف المحلي بمطابقة مباشرة.	user أو None
find_user_by_google_sub	google_sub؛ أول نتيجة ولا None.	user أو None

casefold كتعاون المقارنة بلا حساسية للحروف الكبيرة والصغيرة: Hamza و hamza كيتعاملو هنا كاسم مكرر. البحث ما كيبدلش القيمة المخزنة. "user.get("google_sub" ما كتطلعش KeyError إلا المفتاح غايب.

create_user(username, email, password_hash, *, google_sub=None)

كتعاود تتحقق من تكرار الاسم أو الإيميل، ومن google_sub إلا تعطات. التكرار كيرفع DuplicateUserError. إلا كلشي سليم كتولد id بـ uuid4، كتصاوب القاموس وكتضيفو لـ TEST_USERS، ومن بعد كترجع نفس القاموس. النجمة * كتفرض أن google_sub تتعطى بالاسم. password_hash يقدر يكون None لحساب Google. المفتاح google_sub كيتزاد غير إلا كانت عندو قيمة.

```
user = create_user(
    "Hamza", "hamza@example.com", "PASSWORD_HASH"
)
google_user = create_user(
    "Playera1b2c3d4e", "other@example.com", None,
    google_sub="GOOGLE_USER_ID"
)
```

البحث كيدوز على اللائحة؛ ما كاين لا index لا تخزين دائم. إلا خدمتي بعدة processes، كل process يقدر تكون عندو لائحة مختلفة. قاعدة بيانات مشتركة كتحتاج تصميم منفصل.

google_auth.py: التحقق والربط

(verify_google_token()) و (google_login())

verify_google_token(credential: str) → dict

كتقرا GOOGLE_CLIENT_ID من البيئة. إلا غايب كترد 503. كتستعمل id_token.verify_oauth2_token مع Request () و audience=client_id. المكتبة كتتحقق من التوقيع والصلاحيّة والجهة المصدرة والجمهور المناسب، وكترجع [S1]. [claims]

Request هنا أداة HTTP ديال Google library، ماشي Request ديال FastAPI. TransportError كيتحول لـ 503: التحقق تعطل بسبب الاتصال. ValueError أو GoogleAuthError كيتحولو لـ 401: توكن غير مقبول.

google_login(data: GoogleLoginData) → TokenResponse

أولا كتشغل verify_google_token ف thread. من claims كتأخذ sub، وكتفرض تكون نص غير فارغ. كتقلب على user مرتبط بـ google_sub. إلا لقاتو، كتدور مباشرة لإنشاء توكن الموقع.

إلا ما لقاتوش: كتطلب email كنص غير فارغ، و email_verified تكون True بالضبط. إلا نفس الإيميل مستعمل عند user آخر، كترد 409 وما كتربطش الحسابات أوتوماتيكيا.

كتولد username من Player وزيادة تسعة رموز من uuid4().hex. كتعاود المحاولة إلا الاسم تكرر. كتعيط لـ create_user بالهاش None و google_sub. إلا وقع تكرار أثناء الإنشاء، DuplicateUserError كتولي 409.

شنو كيتخزن وشنو كيرجع؟

كيتخزن id محلي، اسم مولد، google_sub، email، و password_hash=None. ما كيتخزنش Google token فهاد الدالة، وما كتراجعوش للمستخدم. كتخلق JWT محلي باستعمال ["user"]["id"]، ماشي ["claims"]["sub"].

```
# Different identifiers, different issuers:
google_claims["sub"] # Google account identity
user["id"]           # Your local account UUID
site_jwt_payload["sub"] # The local UUID
```

الحساب الموجود بـ google_sub ما كيتحدثش الإيميل ديالو فهاد المسار. وتأكيّد الإيميل عند Google ماشي دائما دليل حديث على ملكية إيميل خارجي تبدل مولاه؛ هادي نقطة خاصها سياسة واضحة فنسخة الإنتاج. [S1]

validation_route.py: أخطاء آمنة

get_route_handler(), AuthRoute(), وsafe_handler()

الأخطاء العادية ديال التحقق تقدر تحمل القيمة اللي المستخدم صيغ. فمسارات المصادقة هاد القيمة تقدر تكون كلمة سر أو توكن. AuthRoute كييف المعالجة باش يرجع مكان الخطأ ونوعو ورسالتو، بلا input الأصلي.

get_route_handler(self) → دالة معالجة

get_route_handler()super() كتجيب المعالج الأصلي ديال FastAPI. كنخزنوه ف original_handler. من بعد كتتعرف دالة داخلية safe_handler، وفالأخر كنرجعوها. كترجع دالة، ما كتنفذش الطلب فداك السطر.

safe_handler(request) → Response

كتستقبل الطلب، وكتنفذ await original_handler(request). إلا نجح، كترجع نفس الجواب. إلا وقع RequestValidationError، كتجمع من كل خطأ غير loc و type و msg، وكترد JSONResponse بحالة 422.

```
# Simplified validation response:
{
  "detail": [
    {
      "loc": ["body", "username"],
      "type": "string_too_short",
      "msg": "String should have at least 3 characters"
    }
  ]
}
```

loc كيبين فين المشكل: هنا username ف type. Body هو تصنيف الخطأ، و msg شرح للمستخدم أو المطور. الشكل التفصيلي للرسائل تابع لنسخة المكتبة، ولكن الحقول المسموحة محددة بالكود.

شنو ما كتشدهش هاد الدالة؟

كتشده RequestValidationError فقط. HTTPException بحال 401 و 409 كتدور للمسار العادي ديال FastAPI. الأعطال الأخرى ما كتتحولش كلها لـ 422. وحتى حذف input من الجواب ما كيغنيش أن logs أخرى فالمشروع محمية تلقائياً.

[[Callable[[Request], Awaitable[Response]]]] وصف للنوع: دالة كتستقبل Request وكتعطي نتيجة خاصها await باش توصل لـ Response. هاد typing كيساعد الفهم والأدوات، ما كيغوضش منطق التحقق.

index.html: الزر ديال Google

كل callback وشنو كيوقع فالواجهة التجريبية

()googleLoaded و ()googleFailed

googleLoaded كتتفعّل ملي Google script يكمل التحميل: كتفك تعطيل زر التهيئة وكتبين رسالة. googleFailed كتبين أن التحميل فشل. بجوج غير كيتحكمو فواجهة التجريب؛ ما كيسجلو حتى حساب فالسيرفر.

الدالة المرتبطة بـ submit

كتنادي event.preventDefault باش الفورم ما يعاودش يحمل الصفحة. كنقرا clientId وكتحديد الفراغات من الجوانب بـ trim. كتراجع واش كيسالي بـ apps.googleusercontent.com. هادي مراجعة بسيطة للشكل، ما كتثبتش صحة الإعداد عند Google.

google.accounts.id.initialize(...)

كتهيئ Google library بـ client_id وcallback. خاص نفس Client ID يكون ف GOOGLE_CLIENT_ID عند السيرفر. Client ID معرف علني ديال التطبيق؛ ظهورو ف HTML طبيعي وماشي بحال JWT_SECRET_KEY.

callback(response)

ملي الدخول ينجح، كتشد response.credential وكتخطو ف textarea. هادا Google ID Token. النص فالصفحة كيطلب منك تنسخو بيدك لـ docs/ ف credential. ما كاينش fetch كينادي auth/google/ فهاد الملف.

()renderButton ومن بعد التهيئة

renderButton كتخلي Google library ترسم زر الدخول. الكود كيغطل زر initialize وكيخلي Client ID للقراءة فقط باش ما يتبدلش وسط التجريب. script ديال Google كيتحمل بـ async، وونload وونerror كييربطو النجاح والفشل بالدوال السابقة.

كيفاش تبان واجهة كاملة؟ مثال إضافي فقط

```
// Illustration: not present in index.html
const res = await fetch("/auth/google", {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify({credential: response.credential})
});
if (!res.ok) throw new Error("Login failed");
const result = await res.json();
// result.access_token is your SITE token.
```

هاد المثال كيفترض نفس origin. إعدادات CORS والنقل الآمن وحماية محاولة الدخول واختيار تخزين التوكن خاصها تكميل قبل واجهة إنتاج؛ مجرد fetch ما كيكملش تصميم الأمان.

مراجع هاد الصفحة: تحليل index.html؛ المثال الأخير توضيحي فقط | الروابط فالآخر

تتبع عملي وحدود النسخة

شنو غادي توقع فعلا، وشنو باقي للتطوير

تتبع حمزة من أول تسجيل

مع 10:00: register كيتحقق، كيحسب الهاش، وكيزيد user. الجواب 201 بلا توكن. مع 10:01: login كيتلقى الحساب وكلمة السر مطابقة، كتولد JWT كيسالي مع 10:31. مع 10:05: me/ كيتحقق من JWT ووجود user، وكترجع معلوماتو.

مع 10:32: me/ كترفض بـ 401 حيث exp فات. إلا السيرفر تعاود تشغل مع 10:10، حتى توكن مازال صالح غادي يلقي TEST_USERS خاوية؛ النتيجة 401 حتى إلا التوقيع بقي صحيح.

الحالة	المعنى	فين كيان هنا؟
201	الحساب تصاوب.	register
200	الطلب نجح.	login و google و me عند النجاح
401	ما قدرناش نثبتو الهوية.	كلمة سر أو توكن مرفوض، أو مستخدم غائب
403	الهوية معروفة لكن العملية ممنوعة.	ما مطابقة ش فهاد routes؛ مثال دور admin
409	تعارض مع حساب موجود.	اسم أو إيميل أو هوية مكررة
422	خاص تصحيح المعلومات.	بيانات الطلب ما احتراماتش النموذج
503	الخدمة غير متاحة لهاد المحاولة.	إعداد Google غائب أو اتصال التحقق تعطل

شنو مازال ما موجودش فالأرشيف؟

قاعدة بيانات دائمة، تأكيد الإيميل للتسجيل العادي، نسيان كلمة السر، Logout، Refresh Token وإلغاء الجلسات، ربط حساب Google بحساب عادي، تحديد محاولات الدخول، MFA، وتدير الصلاحيات حسب الأدوار. حتى إعدادات التطبيق الرئيسي و HTTPS و CORS والنشر ما مبيناش هنا. ما نقدرش نحكم على المشروع كامل من هاد الأرشيف. هاد النقاط حدود للمادة اللي شرحناها، وماشي تغييرات درناها فالكود.

جرب راسك: واش فهمتي المسار؟

أسئلة مفتوحة؛ جاب قبل ما تقلب للصفحة الموالية

- (1) حمزة دار register ونجاحات. مشى مباشرة لـ me/ بلا Authorization. واش الحساب الجديد بوحده كافي باش يدوز؟ علاش؟
- (2) جوج مستخدمين اختارو نفس كلمة السر. واش خاص الهاش ديالهم يكون نفسه باش verify تخدم؟
- (3) شي واحد قرا Payload ديال JWT وبدل sub. واش يقدر يدوز بالتوكن المعدل بلا JWT_SECRET_KEY؟
- (4) حطيتي JWT ف Cookie ولكن خليتي get_current_user كما هي. واش me/ غادي تقراه؟
- (5) المستخدم عندو حساب عادي، ومن بعد دخل بـ Google بنفس الإيميل. شنو كيرجع الكود الحالي؟
- (6) google_sub فيه معرف Google، و"user["id"] فيه UUID محلي. شكون فيهم كيدخل لـ sub ديال JWT الموقع؟
- (7) واش Google ID Token كيغطي موقعك الحق يقرأ رسائل Gmail ديال المستخدم؟
- (8) حذفتي التوكن من الواجهة. واش نسخة من نفس التوكن عند جهاز آخر ولات باطلة تلقائيا؟
- (9) أثناء await run_in_threadpool، واش request ديال التسجيل كمل؟ وواش Event Loop خاصها تبقى واقفة؟
- (10) السيرفر تعاود تشغل، والتوكن باقي عندو عشرين دقيقة. علاش me/ تقدر ترفضو؟
- (11) EmailStr و SecretStr شنو كيضمنو؟ واش الأولى كتعمل Hash والثانية كتأكد ملكية الإيميل؟
- (12) شكون كيغيث لـ register؟ وشكون كيحبب user اللي داخل لـ me؟

تطبيق صغير فالتفكير

خد المخطط ديال Google، وخط صبعك على كل شرط. جرب أربع حالات: توكن منتهي، user معروف بـ google_sub، إيميل مستخدم فحساب عادي، ومستخدم جديد تماما. سمي الدالة اللي خدامة فكل مرحلة.

التصحيح: علاش هادي هي النتيجة؟

رجع للدالة والمخطط، ما تحفظش الجواب بوحده

- (1) لا: register ما كترجعش /me. JWT كحتاج credentials؛ بلا Bearer النتيجة 401.
- (2) لا: Salt مختلف يقدر يعطي هاش مختلف لنفس الكلمة. verify كتستعمل معلومات الهاش المخزنة باش تتحقق.
- (3) لا فهاد التحقق الصحيح: تغيير Payload كيخلي التوقيع ما يوافقش. مجرد قراءة المحتوى ما كتمنحش القدرة على توقيع محتوى جديد.
- (4) لا: HTTPBearer كيقرأ Authorization، ماشي Cookie. خاص تعديل طريقة استخراج التوكن وتصميم الحماية إلا تبدل النقل.
- (5) 409 إلا ما كانش الحساب مربوط بنفس google_sub. ما كاينش ربط تلقائي بالإيميل؛ خاص الدخول بالطريقة الموجودة.
- (6) ["user"]["id"] المحلي هو اللي كيدخل ف JWT الموقع. google_sub كييقي رابط مع هوية Google فالتخزين.
- (7) لا: ID Token لإثبات الهوية. الوصول لـ Gmail API كيحتاج تفويض وصلاحيات وتوكن مناسب منفصل.
- (8) لا: الحذف المحلي ما كيغيرش التوقيع ولا exp. النسخة الأخرى تقدر تبقى مقبولة ما دام المستخدم موجود والشروط متوفرة.
- (9) طلب التسجيل ما كملش؛ كيتسنى نتيجة الهاش. Event Loop تقدر تخدم طلبات أخرى أثناء انتظار worker thread.
- (10) TEST_USERS كتضع مع إعادة التشغيل. get_current_user كتطلب المستخدم الموجود، وما كتكتفיש بتوقيع صحيح.
- (11) SecretStr كتخبي العرض العادي، ماشي EmailStr. Hash. EmailStr كتراجع الصيغة، ماشي ملكية الصندوق. كل وحدة عندها دور محدود.
- (12) FastAPI كينادي register حسب الـ route من بعد التحقق. Depends(get_current_user) كتجيب user لـ me بعد التحقق من التوكن والتخزين.

اختبار أخير بلا حفظ

شرح بصوتك: كلمة السر كتدخل فين؟ كتولي هاش فين؟ JWT كتصاوب فين؟ كتصيفط مع شنو؟ Google كتثبت شنو؟ وإذا شي مرحلة فشلات، شنو الجواب؟ إلا قدرت تربط كل جواب بالدالة ديالو، راه عندك فهم عملي للمسار.

المراجع الرسمية

اضغط على اسم المرجع باش تفتح التوثيق | مراجعة: 20 شتنبر 2026

الشرح ديال سلوك المشروع مبني مباشرة على ملفات backup.zip. المراجع التالية كتوضح المفاهيم العامة؛ أمثلة Google وJWT فالشرح كتميز بين البروتوكول وبين التنفيذ الموجود عندك.

S1 Google: Verify the Google ID token

<https://developers.google.com/identity/gsi/web/guides/verify-google-id-token>

S2 RFC 5849: OAuth 1.0

<https://www.rfc-editor.org/rfc/rfc5849>

S3 RFC 6749: OAuth 2.0

<https://www.rfc-editor.org/rfc/rfc6749>

S4 MDN: Using HTTP cookies

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Cookies>

S5 OpenID Foundation: How Connect Works

<https://openid.net/developers/how-connect-works/>

S6 RFC 7519: JSON Web Token

<https://www.rfc-editor.org/rfc/rfc7519>

S7 RFC 7636: PKCE

<https://www.rfc-editor.org/rfc/rfc7636>

S8 OWASP: Password Storage

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

المراجع الرسمية: تمة

اضغط على اسم المرجع باش تفتح التوثيق | مراجعة: 20 شتنبر 2026

S9 MDN: Set-Cookie

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Set-Cookie>

S10 OWASP: Session Management

https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

S11 OWASP: CSRF Prevention

https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

S12 FastAPI: Dependencies

<https://fastapi.tiangolo.com/tutorial/dependencies/>

S13 FastAPI: Concurrency and async / await

<https://fastapi.tiangolo.com/async/>

S14 MDN: localStorage

<https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>

S15 Pydantic: Secret types

<https://docs.pydantic.dev/latest/concepts/types/#secret-types>

الملفات اللي تشرحو كاملين

,router.py, schemas.py, security.py, current_user.py, memory_store.py, google_auth.py
validation_route.py و index.html. ملفات __pycache__ نواتج تشغيل، ماشي المصدر اللي كنشروه.